20/02/26

Project Component (ECOM F213) Object Oriented Programming

(IC: Anita Agrawal)

Instructions:

Atleast 4 classes should be used to accommodate all the requirements specified in the design problem. Additionally, it should include the following:

1. Nested classes (static or nonstatic, atleast 1)
2. Abstract class (minimum 1)
3. Interface (minimum 1, it can be nested interface or single level or multiple inheritance)
4. Hierarchical Inheritance (atleast 1)
5. Package
6. Exception handling (atleast 3 cases)
7. I/O: File Handling, scanner class etc. (atleast one from any of these)
8. Overloaded methods (minimum 3)
9. Overloaded constructors (minimum 2)
10. Vararg overloading (minimum 1)
11. Wrappers

These are the minimum requirements, but you have the freedom to incorporate a greater number of each as needed.

Note: The names of variables, methods and classes should be lexically rational and should be accompanied with their description in the comments alongside. Your overall code will be something similar to the codes that you write or will be writing in most of your lab hours. It goes without saying that the code should be well indented and should compile and run error free. However, do remember that a non-running code is better than a plagiarized one, and hence the latter one will be penalised heavily if it exceeds 10-15% (may result in recession of this component or the lowest grade). You can use Turnitin for checking the plagiarism. This facility is available in the library. You can contact the library personnels to understand the same. It will not take much time to execute this task.

Draw the UML diagram for the above system. (optional)

Your submission folder should include:

(1) Word doc which explains your project thoroughly along with the UML diagram.
(2) Code appended to the above word doc.
(3) .java files
(4) PPT(you can also show at the time of presentation directly)
(5) Rubrics of the usage of all of the above (1 to 11). Make a table and list the above components and mention how many have you used in each of the category.

Important: Since no weightage has been assigned to this component, letter grades will be assigned which you can include in your resume and the same will be notified to the Placement office by me.

Q1. Smart Campus Management System

Design and implement using Java, a Smart Campus Management System using Object-Oriented Programming (OOP) principles. It should include the following:

1. User Management
The system should support different types of users such as:

- Students
- Faculty members
- Administrative staff
- Security personnel

2. Classroom & Facility Management

- Classroom allocation and scheduling
- Monitoring lights, air conditioning, and equipment status
- Booking shared facilities (auditorium, labs, seminar halls)
- Maintenance scheduling and reporting faults

3. Attendance Management

- Record student and staff attendance
- Generate attendance reports
- Notify absentees or low attendance cases
- Support manual or automated entry methods

4. Energy Management

- Monitor electricity and water usage
- Generate consumption reports
- Detect abnormal usage

5. Security & Access Control

- Digital entry/exit logs
- Visitor management
- Incident reporting
- Alert generation for unauthorized access

6. Reporting & Analytics

- Attendance statistics
- Resource usage reports
- Financial summaries
- Maintenance history

Additionally, your system should support User registration and authentication, Role-based access control, and Profile management and updates.

Q2. Smart Hostel Management System for BITS Pilani Goa Campus

Design and implement a Smart Hostel Management System using Java and Object-Oriented Programming (OOP) principles. It should include the following:

1. Resident Management

- Store student details (name, ID, contact information, course, guardian details).
- Track check-in and check-out information.
- Maintain attendance or entry/exit records.

2. Room Allocation

- Manage room details (room number, type, capacity, occupancy status).
- Allocate and vacate rooms.
- Handle room change requests.

3. Mess and Food Management

- Maintain mess menu schedules.
- Track meal preferences or subscriptions.
- Record feedback or complaints regarding food.

4. Visitor and Security Management

- Record visitor entry/exit logs.
- Generate alerts for unauthorized access.
- Maintain security reports.

5. Fee and Payment Management

- Record hostel fee details and payment status.
- Generate payment reminders.
- Maintain financial reports.

6. Notifications and Alerts

- Notify residents about announcements, maintenance schedules, or emergencies.
- Generate alerts for rule violations or security issues.

Additionally, different users may interact with the system. Your system should provide Administrator access for system control, User access for viewing consumption data and Authentication and role-based permissions.

Q3. Smart Energy Monitoring System in an IT company

Design and implement a Smart Energy Monitoring System using Java and Object-Oriented Programming (OOP) principles. It should include the following:

1. Unit and Device Management

- Add, update, and remove energy consumption units (e.g., rooms, offices, buildings).
- Track individual devices within each unit (lights, ACs, computers, machinery).
- Store device specifications such as power rating and usage type.
- Maintain historical consumption records.

2. Energy Consumption Tracking

- Input daily or hourly consumption readings.
- Automatically calculate total energy usage per unit.
- Display real-time or periodic consumption summaries.
- Maintain historical consumption logs.

3. Reporting System

- Daily, weekly, monthly, or yearly consumption reports.
- Comparison reports between units or time periods.
- Cost estimation based on consumption.
- Graphical or tabular report representation (optional).

4. Usage Trend Prediction

- Identify consumption patterns over time.
- Estimate future energy requirements.
- Highlight peak usage periods.
- Suggest energy optimization measures.

(Simple statistical calculations or basic predictive logic are sufficient.)

5. Alerts and Notifications

- Threshold-based alerts for high consumption.
- Notifications for sudden spikes in usage.
- Maintenance alerts for malfunctioning equipment.
- Alerts for energy-saving recommendations.

6. Energy Efficiency Management

- Identify high-consuming devices.
- Provide recommendations for reducing consumption.
- Track energy-saving initiatives.
- Display efficiency rankings among units.

Different users may interact with the system. Provide Administrator access for system control, User access for viewing consumption data and Authentication and role-based permissions.

Q4. Smart Waste Management System in a residential society

Design and implement a Smart Waste Management System using Java and Object-Oriented Programming (OOP) principles. It should include the following:

1. Waste Bin Management

- Add, update, and remove bins from the system.
- Store bin details such as location, type (organic, recyclable, hazardous), and capacity.
- Track fill level or status (empty, partially full, full).
- Monitor bin maintenance history.

2. Waste Collection Tracking

- Record waste collection events.
- Track which bins were collected and when.
- Maintain collector/vehicle information.
- Store quantity and type of waste collected.

3. Pickup Scheduling

- Generate pickup schedules based on bin status or predefined intervals.
- Adjust schedules dynamically when bins are full.
- Notify collection staff of upcoming pickups.
- Prevent missed or duplicate collections.

4. Recycling and Waste Segregation Management

- Track recyclable vs non-recyclable waste.
- Maintain recycling statistics.
- Record processing details of recyclable waste.
- Encourage segregation through alerts or reports.

5. Alerts and Notifications

- Notify when bins are nearly full or overflowing.
- Alert staff about missed pickups.
- Generate maintenance alerts for damaged bins.
- Provide environmental or efficiency notifications.

6. Reporting and Analytics

- Waste collection reports (daily/weekly/monthly).
- Recycling efficiency reports.
- Pickup performance analysis.
- Bin utilization statistics.
- Environmental impact summaries.

Different users may interact with the system. Administrator access should be given for overall management. Waste collection staff access for operational updates. Optional public user interface for reporting issues. Role-based authentication and permissions.

Q5: Smart Disaster Alert System

Design and implement a Smart Disaster Alert System using Java and Object-Oriented Programming (OOP) principles. It should include the flowing:

1. Environmental Data Monitoring

- Record environmental parameters (temperature, rainfall levels, seismic readings, humidity, wind speed, etc.).
- Store historical environmental data for analysis.
- Support manual data entry or simulated sensor input.
- Organize data based on geographic regions or monitoring zones.

2. Disaster Risk Detection

- Detect abnormal environmental readings.
- Define threshold values for different disaster types (flood, earthquake, heatwave, storm).
- Maintain logs of detected risk events.

3. Prediction and Trend Analysis

- Identify patterns in environmental data.
- Predict possible disasters based on historical trends.
- Highlight high-risk periods or regions.
- Provide preventive recommendations.

(Simple predictive logic or statistical methods are sufficient.)

4. Alert and Notification System

- Generate alerts when risk thresholds are crossed.
- Send notifications to authorities or users.
- Provide severity levels (low, moderate, high, critical).
- Maintain alert history.

5. Reporting and Visualization

- Environmental data reports over time.
- Disaster occurrence summaries.
- Risk assessment reports.
- Optional graphical visualization of trends.

6. Emergency Response Support

- Provide safety guidelines for different disaster types.
- Suggest precautionary measures.
- Maintain emergency contact information.
- Track response actions taken.

Your system should provide Administrator access for system control, Monitoring staff access for data entry and analysis, Public or viewer access for alerts and reports, Role-based authentication.

Q6: Smart Building Management System of a residential area

Design and implement a Smart Disaster Alert System using Java and Object-Oriented Programming (OOP) principles. It should include the flowing:

1. Building Infrastructure Management

- Store details of floors, rooms, departments, or zones.
- Maintain equipment/device records for each area.
- Track infrastructure maintenance history.
- Enable addition, modification, or removal of building components.

2. Lighting Management (for outdoor)

- Track light status (on/off, brightness level).
- Automated control based on time schedules.
- Monitor energy consumption from lighting.

3. Security System Management

- Monitor entry/exit access logs.
- Integrate surveillance or alarm systems (simulation acceptable).
- Detect unauthorized access attempts.
- Maintain incident reports and alerts.

4. Occupancy Monitoring

- Record number of occupants in rooms/zones.
- Analyze peak occupancy periods.
- Optimize electricity, water and other utilities based on occupancy.
- Generate occupancy utilization reports.

5. Alerts and Notifications

- Alerts for equipment failure or abnormal readings, for malfunctioning lights..
- Energy overuse notifications.
- Security breach alerts.
- Maintenance reminders.

6. Reporting and Analytics

- Energy consumption reports.
- Equipment performance analysis.
- Occupancy trends.
- Maintenance and security reports.

Additionally your system should include, Administrator access for full control, Maintenance staff access for equipment updates, Security staff access for monitoring and general user access for viewing information.

Q7: Social Media Event Organizer

Design and implement a Social Media Event Organizer system using Java and Object-Oriented Programming (OOP) principles. The system should allow users to create events, invite friends, manage invitations, track RSVPs, and view event-related information through a console-based interface.

1. **User Management**
   - User registration (username, email, password).
   - Login and logout functionality.
   - View and update user profile.
   - Maintain a friend list.

2. Event Creation and Management
   - Create an event with:
     - Event name
     - Description
     - Date and time
     - Location
     - Event type (public/private)
   - Edit or cancel events (only by event creator).
   - View details of created events.
   - Display list of upcoming and past events.

3. Friend Management

Users should be able to
   - Send and accept friend requests.
   - View friend list.
   - Remove friends.
   - Only friends can be invited to private events.

4. Invitation Management
   - Invite one or multiple friends to an event.
   - Send invitation notifications.
   - View list of invited users.
   - Prevent duplicate invitations.

5. RSVP Tracking
- Invitees can respond with:
  - Accept
  - Decline
  - Maybe
- Event creator can view RSVP status of all invitees.
- Display count of accepted, declined, and pending responses.

6. Notification System

   - Notify users when:
     - They receive an invitation.

    - An event is updated or cancelled.
    - A friend accepts their invitation.
- Display unread notifications.

7. Report Generation:
The system should provide simple analytics.

- Show total events created by a user.
- Display total attendees per event.
- Show upcoming events count.
- Generate basic event summary reports.

Additionally, your system must include a menu driven interface for :

The system should include a menu-driven interface for the following. It can be a simple console based interface:

1. Register / Login
2. Create Event
3. View My Events
4. Invite Friends
5. Respond to Invitation
6. View Notifications
7. Logout

Q8. Hashtag Tracking System

Design and implement a Hashtag tracking system using Java and Object-Oriented Programming (OOP) principles. It should include the following:

1. User Management(basic)

- User registration (username, email, password).
- Login and logout functionality.
- View user profile details.

2. Post Creation and Management

- Create text-based posts.
- Extract hashtags automatically from post content (e.g., words starting with #).
- Store post content along with associated hashtags.
- View all posts created by a user.
- Edit or delete posts.

3. Hashtag Extraction and Storage

- Automatically detect hashtags in a post.
- Store each unique hashtag separately.
- Maintain count of how many times each hashtag is used.
- Avoid counting duplicates within the same post multiple times (optional enhancement).

4. Trending Topic Analysis

- Display top N trending hashtags based on frequency.
- Sort hashtags by usage count (descending order).
- Show frequency statistics for each hashtag.
- Update trending list dynamically when new posts are added.

5. Search and Filter Features

- Search posts by hashtag.
- Display all posts associated with a specific hashtag.
- Show total number of posts using a hashtag.
- Filter hashtags by time period (optional enhancement).

6. Reporting and Analytics

- Display total number of posts.
- Display total unique hashtags.
- Show most active users (based on hashtag usage).
- Generate summary reports.

Additionally, the system should provide a simple menu-driven interface which can be a simple console based interface for the following:

Example menu:

1. Register / Login
2. Create Post
3. View My Posts
4. Search by Hashtag
5. View Trending Hashtags
6. View Reports
7. Logout